

ReactJS Axios GET 請求範例

我們將學習如何使用 Axios HTTP 程式庫構建一個使用 Spring Boot REST API 的 React 應用程式。我們還將使用 Bootstrap 4 來設置 UI 的樣式。

Axios 是一個基於 Promise 的 JavaScript HTTP 客戶端程式庫，可以在瀏覽器和帶有 Node.js 的服務器中使用。Axios 非同步工作，允許您對 REST 服務端點進行 HTTP 調用並使用 JSON REST API。

我們將構建一個簡單的“單頁應用程式”，使用 React 作為前端，使用 spring boot 作為後端。

我們將建立兩個專案：

1. Spring boot 專案-開發簡單的 REST API
2. React App - 使用 Axios 程式庫使用 REST API。

Axios 程式庫概述

Axios 是一個基於 Promise 的 JavaScript HTTP 客戶端程式庫，可以在瀏覽器和帶有 Node.js 的服務器中使用。Axios 非同步工作，允許您對 REST 端點進行 HTTP 調用並使用 JSON REST API。

AXIOS 的特點

請求和回應攔截

簡化的錯誤處理

支援上傳進度

回應超時

取消請求的能力

自動 JSON 數據轉換

下面的代碼基本上是如何發出 HTTP 請求的。

// Make a GET request

```
axios({
  method: 'get',
  url: 'https://api.github.com/users/DemoUser',
});
```

// Make a Post Request

```
axios({
  method: 'post',
  url: '/login',
  data: {
    firstName: 'demo',
    lastName: 'user'
  }
});
```

上面的顯示了我們可以使用 Axios 發出 GET 和 POST HTTP 請求的基本方法。

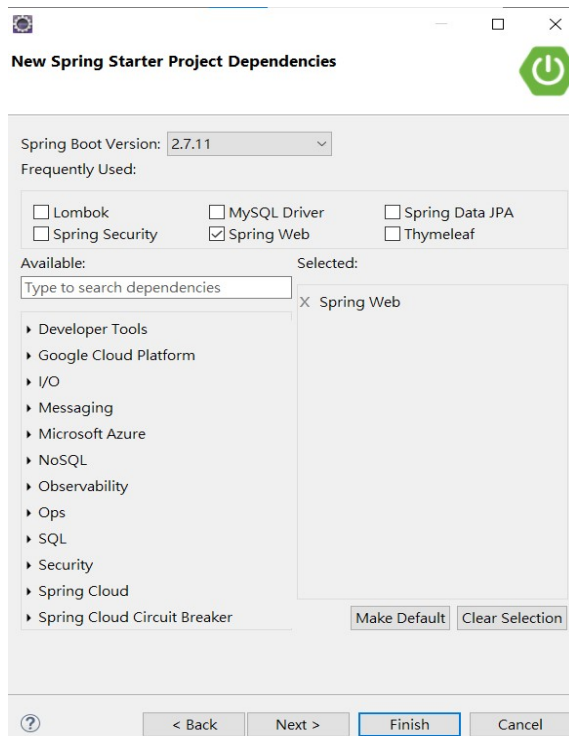
axios 還提供了一組用於執行不同 HTTP 請求的簡寫方法。方法如下：

```
axios.request(config)
axios.get(url[, config])
axios.delete(url[, config])
axios.head(url[, config])
axios.options(url[, config])
axios.post(url[, data[, config]])
axios.put(url[, data[, config]])
axios.patch(url[, data[, config]])
```

開發 Spring Boot 後端應用

讓我們首先使用 Spring Boot 建立一個簡單的 /books rest 端點。

1. 在 Eclipse STS IDE 中建立 Spring Boot 專案



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.7.11</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.example</groupId>
  <artifactId>sbbooks202304</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>sbbooks202304</name>
  <description>Book project for Spring Boot</description>
  <properties>
    <java.version>11</java.version>
  </properties>
  <dependencies>
    <dependency>
```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

BookController.java

```

import java.util.Arrays;
import java.util.List;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

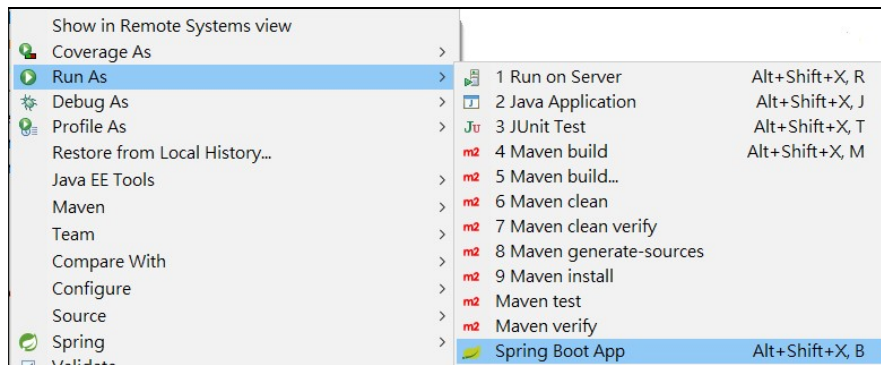
import com.example.demo.model.Book;

@RestController
public class BooksController {
    @CrossOrigin(origins = "http://localhost:3000")
    @GetMapping("/books")
    public List<Book> getBooks() {
        return Arrays.asList(new Book(1, "Core Java", "Amy"), new Book(2,
"Effective Java", "John"), new Book(3, "Head First Java", "Tony"));
    }
}

```

Book Model:

```
public class Book {  
    private int id;  
    private String bookName;  
    private String author;  
    public Book(int id, String bookName,String author) {  
        this.id = id;  
        this.bookName = bookName;  
        this.author=author;  
    }  
  
    public int getId() {  
        return id;  
    }  
  
    public String getBookName() {  
        return bookName;  
    }  
  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    public void setBookName(String bookName) {  
        this.bookName = bookName;  
    }  
  
    public String getAuthor() {  
        return author;  
    }  
  
    public void setAuthor(String author) {  
        this.author = author;  
    }  
}
```



瀏覽器中 <http://localhost:8080/books> 鏈接將顯示 JSON 格式的書籍列表。

開發 React JS 前端應用程序

讓我們繼續建立一個 React 應用程序來使用以下端點：

<http://localhost:8080/books>

我們將使用瀏覽器內置的 `fetch()` 函數在 React App 中進行 REST API 調用。

讓我們從使用 `create-react-app` CLI 建立 React App 開始。

1 - 使用 Create React App 建立 React UI

Create React App CLI 工具是官方支持的建立單頁 React 應用程序的方法。它提供了一個沒有配置的設置。

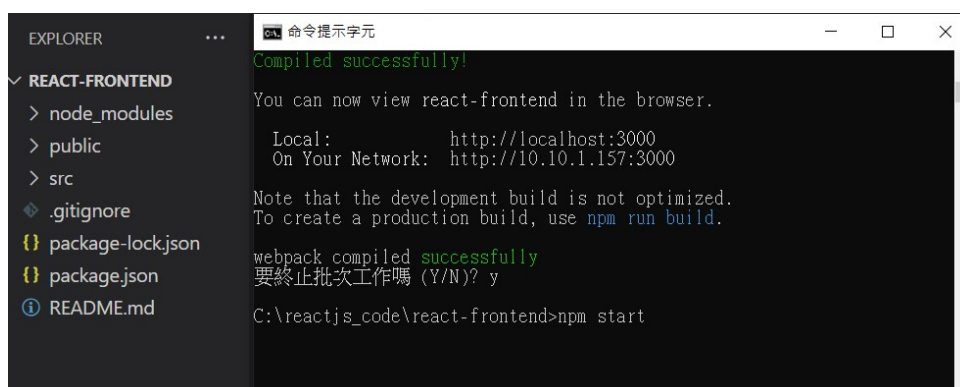
要建立新的應用程序，您可以選擇以下方法之一：

使用 `npm`

`npm create-react-app react-frontend`

```
命令提示字元
C:\reactjs_code>npm create-react-app react-frontend
Creating a new React app in C:\reactjs_code\react-frontend.
Installing packages. This might take a couple of minutes.
Installing react, react-dom, and react-scripts with cra-template...
added 1423 packages in 35s
```

使用 VSCode 開啟使用 `react-frontend` 資料夾



讓我們探索 **React** 項目的重要文件和文件夾。

對於要建立的專案，這些文件必須具有準確的文件名：

public/index.html 是頁面模板；
src/index.js 是 **JavaScript** 入口點。

您可以刪除或重命名其他文件。
讓我們快速瀏覽一下項目結構。

package.json - **package.json** 文件包含我們的 **React JS** 項目所需的所有依賴項。最重要的是，您可以檢查您正在使用的 **React** 的當前版本。它具有啟動、建置我們的 **React** 應用程序的所有腳本。

public 文件夾 - 公用文件夾包含 **index.html**。由於 **React** 用於構建單頁應用程序，因此我們有一個 **HTML** 文件來呈現我們所有的組件。基本上，它是一個 **HTML** 模板。它有一個以 **id** 為根的 **div** 元素，我們所有的組件都在這個 **div** 中呈現，**index.html** 作為完整的 **React** 應用程序的單個頁面。

src 文件夾- 在此文件夾中，我們擁有所有全域 **javascript** 和 **CSS** 文件。我們將建立的所有不同組件都放在這裡。

index.js - 這是你的 **React** 應用程序的渲染器。

node_modules - **NPM** 或 **Yarn** 安裝的所有套件程式庫都將駐留在 **node_modules** 文件夾中。

App.js - **App.js** 文件包含我們的 **App** 組件的定義，它實際上在瀏覽器中呈現，這是根組件。

使用 **NPM** 在 **React** 中添加 **Bootstrap**

打開一個新的命令提示字元窗口，引導到專案的文件夾，然後運行以下命令：

npm install bootstrap --save

安裝引導程序包後，您需要將其導入到您的 **React** 應用程序入口文件中。
打開 **src/index.js** 文件並添加以下代碼：

```
Import "bootstrap/dist/css/bootstrap.min.css";
```

安裝 axios 客戶端

讓我們在根文件夾中的以下命令在我們的 React 專案中安裝 Axios 程式庫：

```
npm install --save axios
```

React 服務組件——REST API 調用

讓我們建立 apiService.js 服務實作以通過 Axios 進行 HTTP REST 調用
我們的後端 USER 端點位於 <http://localhost:8080/books>

```
import axios from 'axios';
```

```
const BOOK_API_REST_URL = "http://localhost:8080/books";
```

```
class APIService {
```

```
  getBooks(){  
    return axios.get(BOOK_API_REST_URL);  
  }
```

```
}
```

```
export default new APIService();
```

建立 class 元件 BookComponent.js

```
import React from 'react'
```

```
import APIService from './apiServices'
```

```
export default class BookComponent extends React.Component {
```

```
  constructor(props) {  
    super(props)  
    this.state = {  
      books: []  
    }  
  }
```

```
}
```



```

componentDidMount(){
  ApiService.getBooks().then((rs) => {
    this.setState({ books: rs.data })
    console.log(this.state.books);
  })
  .catch(function (ex) {
    console.log('Response parsing failed. Error: ', ex);
  });
}

render() {
  return (
    <div>
      <h2 className="text-center">Book Details</h2>
      <table className="table table-striped">
        <thead>
          <tr>
            <th>Book Id</th>
            <th>Book Name</th>
            <th>Book Author</th>
          </tr>
        </thead>
        <tbody>
          {
            this.state.books.map(book =>
              <tr key={book.id}>
                <td>{book.id}</td>
                <td>{book.bookName}</td>
                <td>{book.author}</td>
              </tr>
            )
          }
        </tbody>
      </table>
    </div>
  )
}

```

```
}
```

讓我們解釋上面的程式碼。

constructor() - 在安裝組件之前執行 **constructor()** 。在建構函數中，我們宣告了 **book** 狀態變數：

```
constructor(props) {  
    super(props)  
    this.state = {  
        books: []  
    }  
}
```

componentDidMount() - **componentDidMount()** 會在組件安裝並準備就緒後立即執行。如果您需要從遠程端點載入數據，這是啟動 **API** 調用的好地方。請注意，我們正在從 **componentDidMount()**方法調用 **APIService** 的 **loadBooks()** 方法：

```
componentDidMount(){  
    APIService.getBooks().then((rs) => {  
        this.setState({ books: rs.data })  
        console.log(this.state.data)  
    })  
    .catch(function (ex) {  
        console.log('Response parsing failed. Error: ', ex);  
    });  
}
```

render() - **render()** 方法是最常用的生命週期方法。你會在所有 **React** 類別中看到它。這是因為 **render()** 是 **React** 類別組件中唯一需要的方法。實際將 **HTML** 輸出到 **DOM** 的 **render()**方法。

APIService - 我們使用 **APIService** 類別來獲取進行 **API** 調用的 **getBooks()** 方法。我們正在使用 **map 運算子** 來遍歷我們的圖書列表並創立如下檢視：

```
{  
    this.state.books.map(book =>  
        <tr key={book.id}>  
            <td>{book.id}</td>  
            <td>{book.bookName}</td>  
            <td>{book.author}</td>  
        </tr>  
    )  
}
```

```
}
```

```
index.js
```

```
import React from 'react';
```

```
import ReactDOM from 'react-dom/client';
```

```
import './index.css';
```

```
import reportWebVitals from './reportWebVitals';
```

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

```
import BookComponent from './BookComponent';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(  
  <React.StrictMode>
```

```
    <BookComponent />
```

```
  </React.StrictMode>
```

```
);
```

```
// If you want to start measuring performance in your app, pass a function
```

```
// to log results (for example: reportWebVitals(console.log))
```

```
// or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
```

```
reportWebVitals();
```